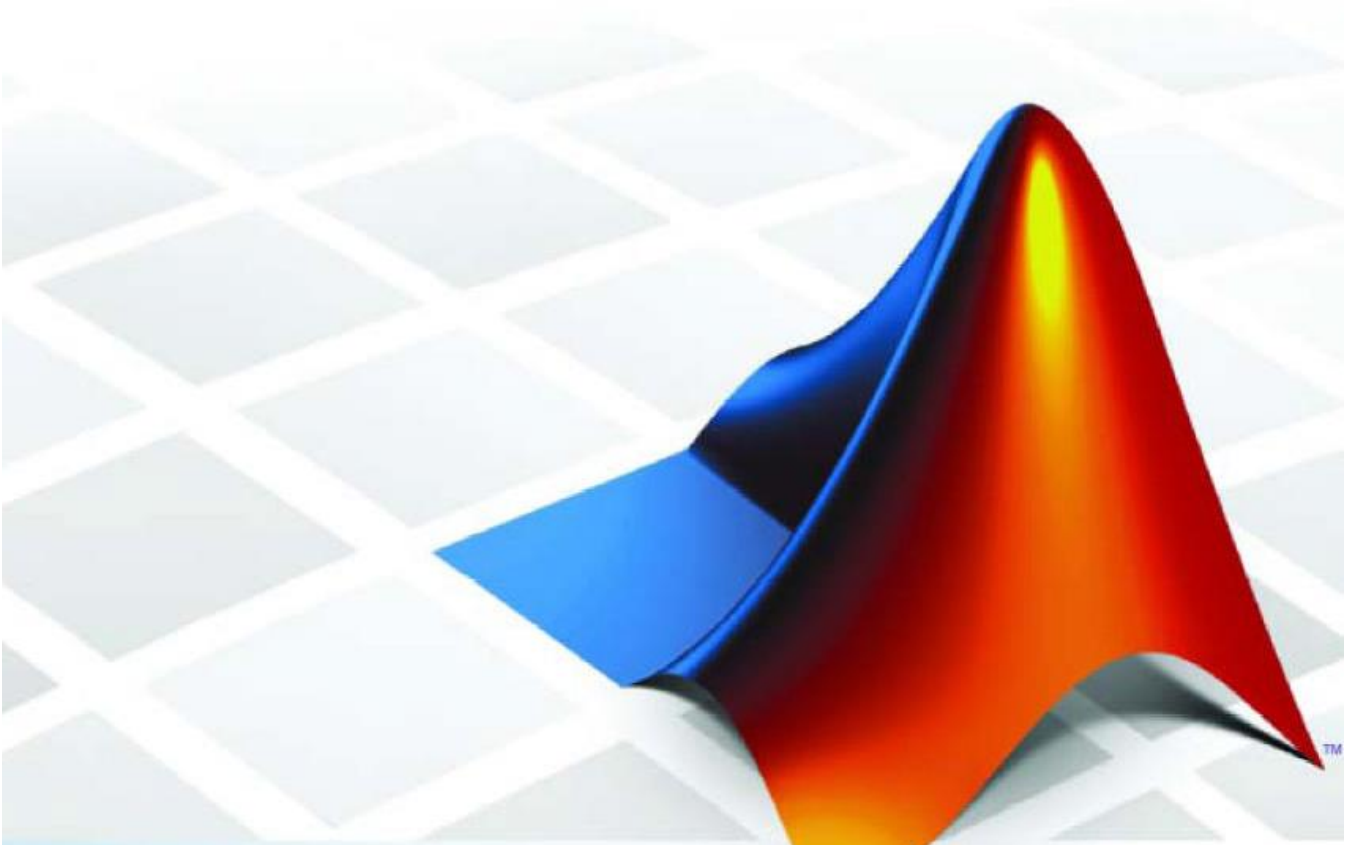


MATLAB®

The MathWorks™

Accelerating the pace of engineering and science

PART-1
Introduction to MATLAB

Exercise Objective:

Familiarization with MATLAB screen

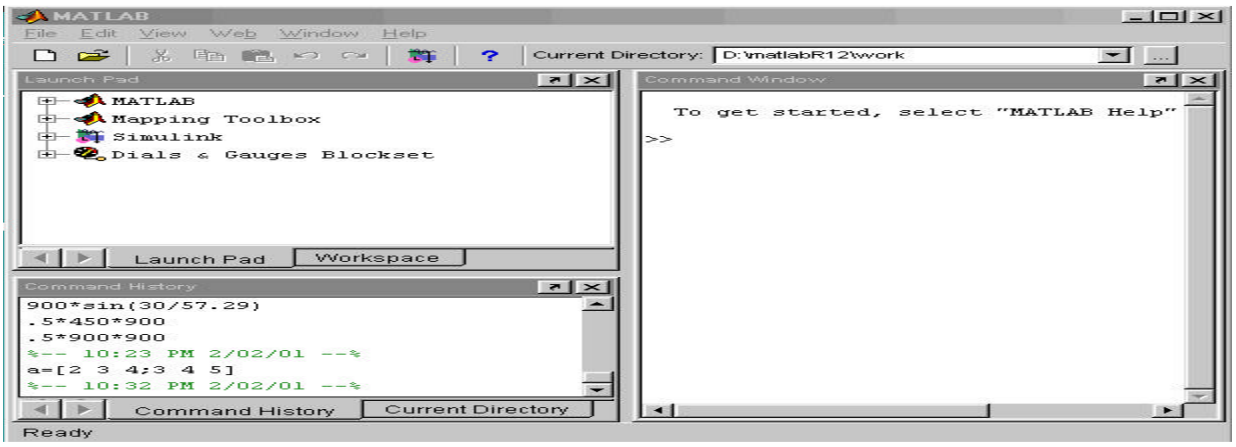
A minimal MATLAB Session

Theory:

MATLAB is a high-performance language for technical computing. It integrates computation, visualization and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. It is particularly convenient for modeling, simulation, analysis and design of dynamic systems.

The purpose of this introductory lab is to give you a quick introduction to MATLAB enabling you to use the software from the very beginning of the Signal & Systems course. To run MATLAB, double-click on the MATLAB icon. Once the program is loaded you will see the prompt">>" in the command window. The command window is the main window in which we communicate with the MATLAB interpreter. Anything typed after "%" sign is treated as a comment and ignored by MATLAB.

Procedure:



MATLAB has the following basic window components:

- **Launch Pad Window**

• To access all MATLAB services and toolboxes

- **Command Window**

- To execute commands in the MATLAB environment

- **Current Directory Window**

- To quickly access files on the MATLAB path

• **Figure Window**

- To display graphical output from MATLAB code

Basic Components of the MATLAB Environment

• Workspace Window

- to view variable definitions and variable memory allocations

• M-File Editor/Debugger Window

- to write M-files (includes color-coded syntax features)
- to debug M-files interactively (break points)

• MATLAB Path Window

- to add and delete folders to the MATLAB path

• Command History Window

- displays all commands issued in MATLAB since the last session (good for learning and verification)

MATLAB Command Window

• The command window allows you to interact with MATLAB just as if you type things in a calculator

• Cut and paste operations ease the repetition of tasks

• Use 'up-arrow' key to repeat commands (command history)

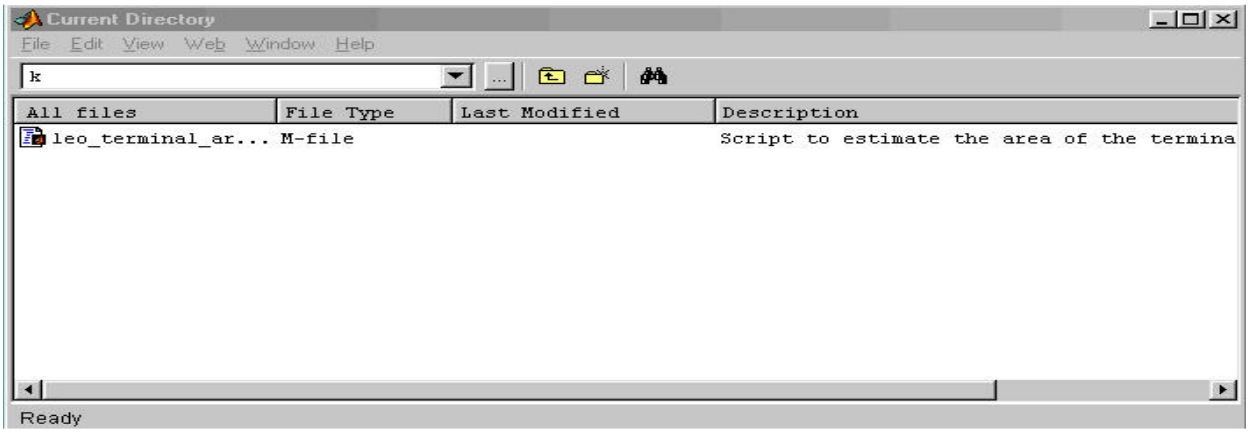
MATLAB Launch Pad Window

• The launch window allows you to quickly select among various MATLAB components and toolboxes

MATLAB Current Directory Window

• Provides quick access to all files available in your Path

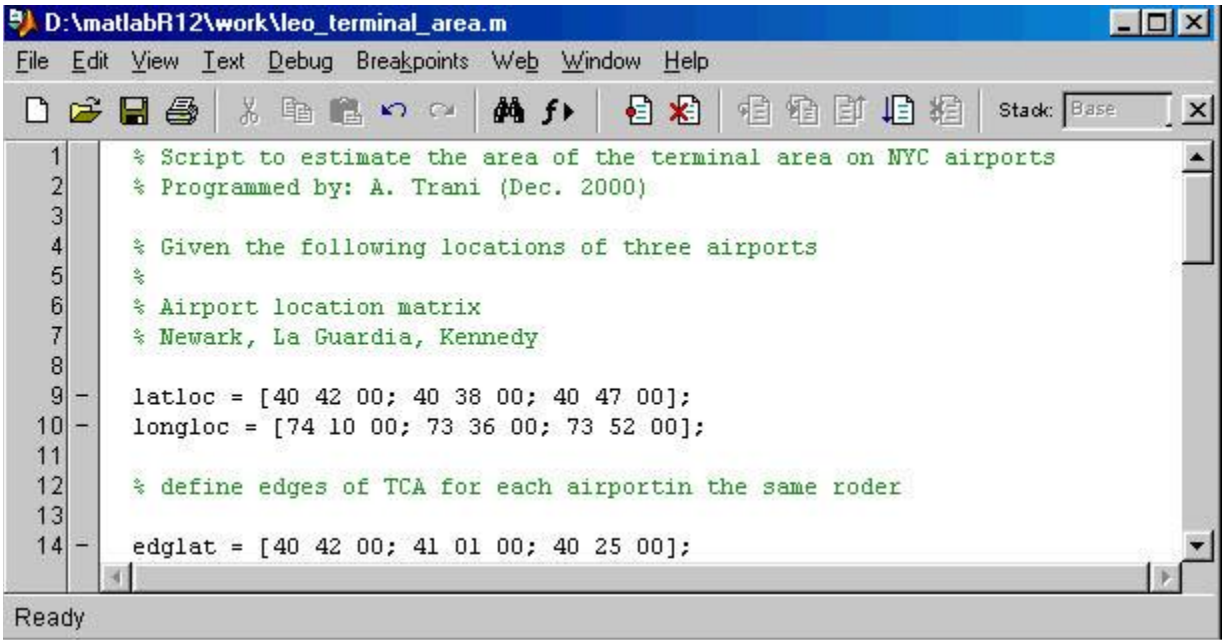
• Provides a brief description (when files are commented out) of each M-file



MATLAB Editor/Debuger Window

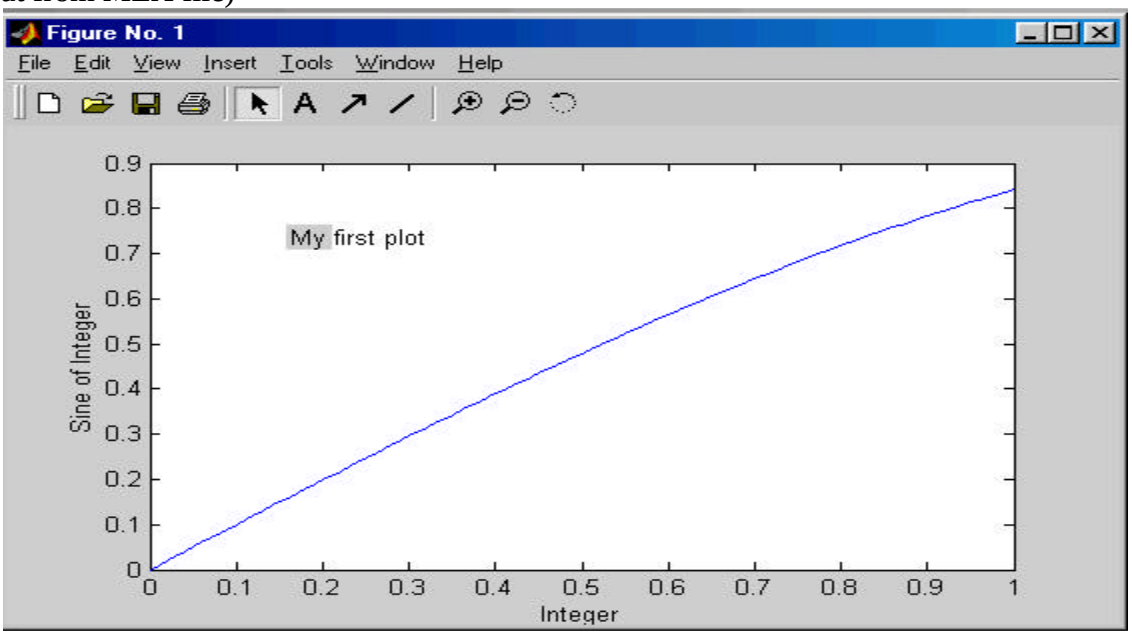
• Provides the same functionality found in most programming language development environments

- Color codes MATLAB built-in functions (blue color)
- Easy access to cut, paste, print, and debug operations
- Checks balance in MATLAB function syntax



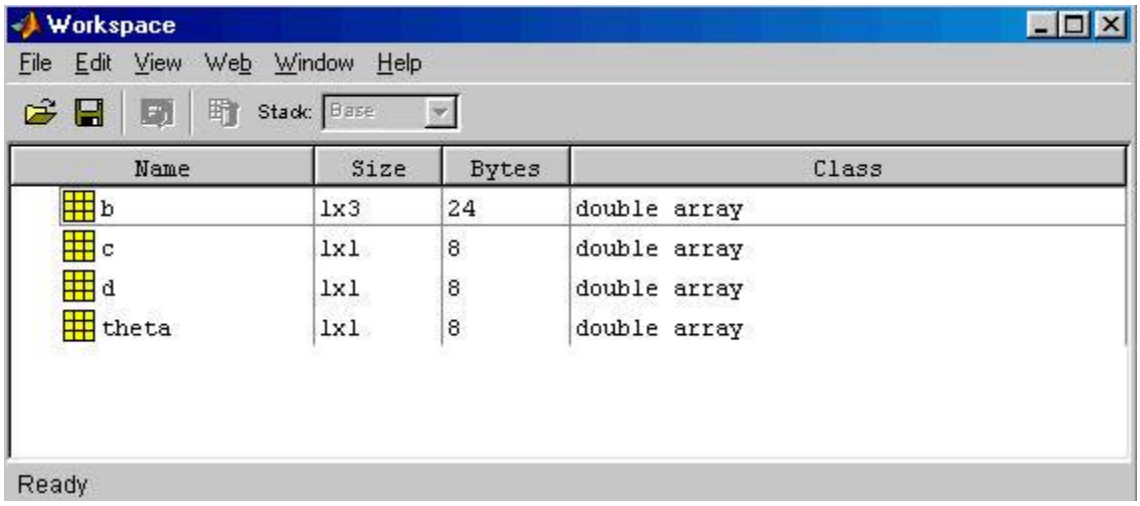
MATLAB Figure Window

• Displays the graphic contents of MATLAB code (either from Command Window, an M-file, or output from MEX file)



MATLAB Workspace

As you develop and execute models in MATLAB the workspace stores all variables names and definitions for you. All variables are usually available to you unless the workspace is clear with the '>>clear' command.



Getting Started: -

If you don't know anything about MATLAB, then MATLAB help is the best way to learn about something. In the command window of MATLAB simply write 'help';

>> help

It will display list of all toolboxes included in MATLAB. Then by investigating the name of toolbox or the name of a function, which you would like to learn how to use, use the 'help' command:

>> help function name

This command displays a description of the function and generally also includes a list of related functions. If you cannot remember the name of the function, use the 'lookfor' command and the name of some keyword associated with the function:

>>look for keyword.

This command will display a list of functions that include the keyword in their descriptions. MATLAB also contains a variety of demos that can be with the 'demo' command.

MATLAB as a calculator

```
>>1+2*3      >>(1+2)*3
Ans = 7        ans = 9
```

Order of precedence is from left to right and from inner most parenthesis to outermost

Declaring and assigning a variable in MATLAB

Variable name=value(or expression)

Default variable is and

Let x=1;

Y=3*x

Y=3

Suppressing output:

If you simply type a statement and press **Return** or **Enter**, MATLAB automatically displays the results on screen. However, if you end the line with a semicolon, MATLAB performs the computation but does not display any output. This is particularly useful when you generate large matrices

Multiple statements per line with,and; operator

e.g>>x=1;y=5;z=9
ans
z=9

Complex Numbers:

MATLAB also supports complex numbers.The imaginary number is denoted with the symbol I or j. There are different methods to declare a complex number which are given below

```
>>com=4+2i
        com=4.0000 + 2.0000i
>> c=complex(1,-2)
        c=1.0000-2.0000i
```

Conjugate:

```
>> c=conj(com)
        c = 4.0000 – 2.0000I
```

Real Part:

```
>> r=real(com)
        r = 4
```

Magnitude:

```
>>mag=abs(com)
        mag = 4.4721
```

Imaginary Part:

```
>> i=imag(com)
        i = 2
```

Angle in Radian:

```
>>a_radian = angle(com)
        a_radian = 0.4636
```

Angle in Degree:

```
>>a_degree = angle(com)*180/pi
        a_
```

Built-in Mathematical functions in MATLAB

MATLAB comes with a large number of built-in functions that operate on matrices on an element-by element basis. These include.

sin sine

cos cosine
tan tangent
asin inverse sine
acos inverse cosine
atan inverse tangent
exp exponential
log natural logarithm
log10 common logarithm
sqrt square root
abs absolute value
signsignum

for further functions see help elfun

helpspecfun

Before quitting,**Save** .m files in current work (default) directory of MATLAB

What do the following commands do? Explain them by running these commands in your MATLAB!

- clc
- clear all
- clear x
- clear x,y
- clear('x')
- close all
- who
- whos
- disp

Explain the following Logical Operators in your own words.

- &
- &&
- |
- ||
- ~=
- ==
- <, >, <=, >=

For Loop:

The execution of a command is repeated at a fixed and predetermined number of times. Syntax is

```
for     variable = expression
        statements
end
```

e.g

```
for     i = 1:5

        x= i * i

end
```

While Loop:

This loop is used when the number of passes is not specified. The looping continues as long as stated condition is satisfied. The while loop has the form:

```
while   expression
        statements
end
```

e.g

```
x = 1
while   x <= 10
        x = 3*x
end
```

Note: *Other control statements include* ***break, continue, return, switch****… etc. Every student should be able to use all these control statements.*

Task 1: Write down the result of the following "For Loop".

```
>> v= 1:3:10

    >>for j=1:4,

        V (j) =j;

    end
```

Output:

Task 2: Write down the result of the following While Loop.

```
j=1;
while(j<=4)
a(j)=10*j;
j=j+1;
end
a
```

Task 3: Comment on the below line of code and write the output.

```
a=1:10
i=1;
while(i<=10)
b(i)=2^a(i);
i=i+1;
end
b
```

Task 4: Write a program for the Table of 5 that will start in reverse order i.e., 5*10 to 5*1

Task 5: Write a program that will generate following outputs. Hint(cumsum)

1st Iteration= 5*1

2nd Iteration=5*3

3rd Iteration= 5*6

4th Iteration= 5*10

5th Iteration= 5*15

6th Iteration= 5*21

7th Iteration= 5*28

Task 6: Write down at least four programs of your desire using above logical operators.

To end a session

>>exit or quit

Conclusion:

_____

_____

_____

PART-2
Introduction to Matrices and Arrays

Exercise Objective:
To become familiar with matrices, arrays, and operations that can be performed on them.

Theory:

MATLAB stores variables in the form of matrices, which are $M \times N$, where M is the number of rows and N the number of columns. A 1×1 matrix is a scalar; a $1 \times N$ matrix is a row vector, and $M \times 1$ matrix is a column vector. You can enter matrices into MATLAB in several different ways:
• Enter an explicit list of elements.

• Generate matrices using built-in functions etc.

Procedure: To enter the matrix

Real scalar >> x = 5
Complex scalar >> x = 5+10j (or >> x = 5+10i)
Row vector >> x = [1 2 3] (or x = [1, 2, 3])
Column vector >> x = [1; 2; 3]
3 × 3matrix >> x = [1 2 3; 4 5 6; 7 8 9]

Generating Matrices using Function:
MATLAB software provides two major functions that generate basic matrices.
1-zeros All zeros
2-ones All ones
Here are some examples:

```
>> a = ones(3,3) % Identity Matrix

A =

     1   1   1

     1   1   1
```

Unit Matrix
The n × n identity matrix is a matrix of zeros except for having ones along its leading diagonal (top left to bottom right). This is called eye (n) in MATLAB.

```
>>a(1,3) = 3   % Assigning a value

A =

     0   0   3   0

     0   0   0   0

     0   0   0   0

     0   0   0   0
```

```
>>a=eye(3)

A =   1   0   0
      0   1   0
      0   0   1
```

Diagonal matrix

A diagonal matrix is similar to the identity matrix except that its diagonal entries are not necessarily equal to 1.

```
>>d=[1 4 7];
>>D=diag(d)
D=
      1   0   0
      0   4   0
      0   0   7
```

Generating vectors

Vectors can be generated using the ':' command. For example, to generate a vector *x* that takes on the values 0 to 10 in increments of 0.5, type the following which generates a 1×21 matrix

```
>> x = [0:0.5:10];
```

Other ways to generate vectors include the commands: 'linspace' which generates a vector by specifying the first and last number and the number of equally spaced entries between the first and last number, and 'logspace' which is the same except that entries are spaced logarithmically between the first and last entry

Accessing vector elements

Elements of a matrix are accessed by specifying the row and column. For example, in the matrix specified by A = [1 2 3; 4 5 6; 7 8 9], the element in the first row and third column can be accessed by writing

```
>> x = A(1,3) which yields 3
```

The entire second row can be accessed with

```
>> y = A(2,:) which yields [4 5 6]
```

where the ':' here means "take all the entries in the column". A submatrix of *A* consisting of rows 1 and 2 and all three columns is specified by

```
>> z = A(1:2,1:3) which yields [1 2 3; 4 5 6]
```

Arithmetic matrix operations

The basic arithmetic operations on matrices (and of course scalars which are special cases of matrices) are:

+ addition
- subtraction
* multiplication
/ right division
\ left division
^ exponentiation (power)
' conjugate transpose

Arithmetic operations on matrices

Addition:

>> d =a+b

d=6 8 10 12

Subtraction:

>> e =a-b

e=-4 -4 -4 -4

Multiplication:

>> g =a*c g=70

Division:

>> h=a/b

h=0.4023

Assigning an Element from a Matrix:

>> i=a(1,3) % will assign elements in row 2 and columns 3 of matrix "a "to variable "i".

Note: An error message occurs if the sizes of matrices are incompatible for the operation.
The difference between matrix multiplication and element-by-element multiplication is seen in the following example.

```
>>A = [1 2; 3 4]
A =
1 2
3 4
>>B=A*A
B =
7 10
15 22
>>C=A.*A
C =
1 4
9 16
```

The magic Function

MATLAB has a built-in function that creates magic squares of almost any size. Not surprisingly, this function is named magic:

```
B = magic (4)
B =
16 2 3 13
5 11 10 8
9 7 6 12
4 14 15 1
```

Properties of MAGIC SQUARE:

You are probably already aware that the special properties of a magic square have to do with the various ways of summing its elements. If you take the sum along any row or column, or along

either of the two main diagonals, you will always get the same number. Let us verify that using MATLAB.
The first statement to try is

```
sum(A)
```

MATLAB replies with

```
ans =
34 34 34 34
```

```
A'
```

Produces sum of rows

```
ans =
16 5 9 4
3 10 6 15
2 11 7 14
13 8 12 1
```

and

```
sum(A')'
```

produces a column vector containing the row sums

```
ans =
34
34
34
34
```

The sum of the elements on the main diagonal is obtained with the sum and the diag functions:

```
diag(A)
```

produces

```
ans =
16
10
7
1
```

and

```
sum(diag(A))
```

produces

```
ans =
        34
```

Linear algebra functions on Matrices

Matrices can sometimes be inverted:
>>inv(B)

Note: A matrix can be inverted if and only if its determinant is nonzero: If you try to compute the inverse for a matrix with non-zero determinant you will get a warning message:
Warning: Matrix is close to singular or badly scaled.
The results may be inaccurate.

>>det(B);

Arrays

When they are taken away from the world of linear algebra, matrices become two-dimensional numeric arrays. Arithmetic operations on arrays are done element by element. This means that addition and subtraction are the same for arrays and matrices, but that multiplicative operations are different. MATLAB uses a dot, or decimal point, as part of the notation for multiplicative array operations.
The difference between matrix multiplication and element-by-element multiplication is
seen in the following example.

```
>>A = [1 2; 3 4]
A =
1 2
3 4
>>B=A*A
B =
7 10
15 22
>>C=A.*A
C =
1 4
9 16
```

Cell:

A cell *array* is a data type with indexed data containers called cells, where each cell can contain any type of data. Cell arrays commonly contain either lists of text, combinations of text and numbers, or numeric arrays of different sizes.

```
cell{1}=rand(4,6)
```

Task 1: Comment on the below line of code and write the output.

```
C = {1,2,3;'text', rand(2,3,2),{11; 22; 33}}
```

Task 2: Run the magic(3,4) command and also write the output of it. If Error occur, explain its reason?

Task 3: What do the following commands do? If A is vector and Matrix

- max(A)
- min(A)
- diff(A)
- sum(A)
- sum(sum(A))
- v(1:3)
- v(3: end)
- v(:)
- v(3,3) = 0
- A(2,:)

- A(:,2:3)

Task 4: What does the following command do?

- linspace(a,b,n)
- rand(a,b)
- round(x)
- ceil(x)
- floor(x)
- fix(x)
- a=randi(2,4)
- a=randi(2,4,23)
- a=randi([1 10],2,6)
- length(a)
- size(a)
- ones(a,b)
- zeros(a,b)
- b=cumsum(a)
- b=cumsum(a,1)
- b=cumsum(a,1,’reverse’)
- b=cumsum(a,2)
- b=cumsum(a,2,’reverse’)

Task5: Explain the output of the following commands?

A = [1 2 3; 4 5 6; 7 8 9]

B = A([2 3],[1 2])

Task 6: Differentiate between vector, array and matrix

Other query commands

We can get the size (dimensions) of a matrix with the command size:
Size(B)
Length(B) etc

Conclusion:

_____
_____
_____

PART-3
Introduction to MATLAB

Exercise Objective:

PART A: Dealing with Polynomials & Partial Fraction Expansion
PART B: Plotting in MATLAB
PART C: Solving ordinary differential equations (ODE) symbolically.

Theory:

MATLAB provides a full programming language that enables you to write a series of MATLAB statements into a file and then execute them into a single document.

Polynomials are used for the analysis of transfer function because in case of transfer function we want to see output and input in the form of polynomial. So it is important to study the polynomials in MATLAB.

When trying to find the inverse Laplace transform (or Z transform) it is helpful to be able to break a complicated ratio of two polynomials into forms that are on the Laplace transform or Z transform table. We will illustrate here using Laplace transforms. This can be done using the method of Partial fraction expansion (PFE) which is the reverse of finding a common denominator and combining fractions. It is possible to do PFE by hand also. We will illustrate hand computation only for the simplest case when there are no repeated roots, and the order of the numerator polynomial is strictly less than the order of denominator polynomial.

The primary tool we will use for plotting in MATLAB is *plot()*.The MATLAB environment provides a wide variety of techniques to display data graphically.
The type of graph you choose to create depends on the nature of your data and what you want to reveal about the data. You can choose from many predefined graph types, such as line, bar, histogram, and pie graphs as well as 3-D graphs, such as surfaces, slice planes, and streamlines. After you create a graph, you can extract specific information about the data. You can also edit graph components as well as annotate graphs. Annotations are the text, arrows, callouts, and other labels added to graphs to help viewers see what is important about the data. Lastly you can print your graph on any printer connected to your computer.

Though MATLAB is primarily a numeric package, it can certainly solve straightforward differential equations symbolically as well using Math symbolic toolbox as demonstrated in this lab.

PART A: Dealing with Polynomials

Polynomials

Polynomials arise frequently in systems theory. MATLAB represents polynomials as row vectors of polynomial coefficients. For example, the polynomial $s2 + 4s - 5$ is represented in MATLAB by the polynomial >> p = [1 4 -5]. The following is a list of the more important commands for manipulating polynomials.

roots(p) Express the roots of polynomial *p* as a column vector.
polyval(p,x) Evaluate the polynomial *p* at the values contained in the vector *x*
conv(p1,p2) Computer the product of the polynomials *p*1 and *p*2
deconv(p1,p2) Compute the quotient of *p*1 divided by *p*2
poly2str(p,'s') Display the polynomial as an equation in*s*

Declaring a Polynomial:

```
>> coeffs(poynomial)

>> pol1= [1 2 4];

>> pol2= [1 4 8];
```

Automatic declare the Polynomial=x^3+x+3

```
pol1=coeffs(polynomial,'all')  % coefficents should be in higher order

pol2=fliplr(poly)

pol3=flipud(poly)

R=roots(pol3)
```

Roots of Polynomial:

```
>> rt_pol1=roots (pol1)

rt_pol1 =

  -1.0000 + 1.7321i

  -1.0000 - 1.7321i
```

Determining Coefficients from given Roots:

```
>> c=poly(rt_1)

>> c=poly(rt_pol1)

c = 1.0000    2.0000    4.0000
```

Finding value of Polynomial at a given point:

```
e=polyval(pol1,2)

e =12
```

Derivative of Polynomial function:

```
>> pol1=[1 2 4];

>> pol2=[1 4 8];

>> h=polyder(pol1)

h =  2     2
```

or h=polyder(pol1,pol2)

Division of two Polynomials

```
>>num=[2 5 4 6 4];

>>den=[6 66 35 2];

>> [q r]=deconv(num,den)

q =

   0.3333   -2.833

r =

       0    0.0000  179.3333  104.5000    9.6667
```

Integration of Polynomial Function

```
>> pol1=[1 2 4];

>> pol2=[1 4 8];

>> k=polyint(pol1)

k = 0.3333    1.0000    4.0000         0

>> m=polyint(pol1,pol2)

m =    0.3333    1.0000    4.0000    1.0000    4.0000    8.0000

j =  4    18    40    32
```

Determine roots of the following functions using MATLAB

Task: x^4 + 7x^3 +15x^2 – 25x + 8

Task: x^2 -18x +25

Determine value of polynomial at given instant using MATLAB

Task: x^5+2x^4-3x^3+7x^2-5x+7 at x=2

Task: x^3-2x^2-5x+17 at x=2

Dealing with Partial Fraction expansion:

[R, P, K] = RESIDUE(B,A) finds the residues, poles and direct term of a partial fraction expansion of the ratio of two polynomials B(s)/A(s).

[B, A] = RESIDUE(R,P,K), with 3 input arguments and 2 output arguments, converts the partial fraction expansion back to the polynomials with coefficients in B and A.

Example: % given a ratio of polynomials G(s) = (3s + 7)/(s^3 + 6s^2 + 11s + 6),determine the corresponding PFE

% this is done by first defining the numerator and denominator polynomials

```
>> num1= [3 7].

>> den1= [1 6 11 6].

>> [r, p, k] =residue(num1, den1) % r is the vector of PFE coefficients is the vector of roots of den1 and k=0 if the degree of num1 < degree of den1

r =

   -1.0000

   -1.0000

    2.0000

p =

 -3.0000

   -2.0000

   -1.0000

k =

 []
```

Solve the following functions using PFE method.

Task: 3/(s^3 + 3s +2)

Task: s + 3/s^3 + 5s + 4

Solve the Polynomial Equation by using solve command

syms x

y=x^2 +x+4

solve(y,x)

Derivative of an Equation

diff(y,x)

Integration of an Equation

int(y,x)

Sum of Series in MATLAB

Syntax=symsum(f,k,a,b) where f is the function, k is with respect to limit, a is lower limit and b is the upper limit.

F1= $\sum_{k=0}^{10} k$^2

Matlab Code

symsum(k^2,k,0,10)

Task: Write the output and Matlab Code for the following series

- $\sum_{k=1}^{5} 1/k$^2

- $\sum_{k=0}^{5} k\text{^}2$

- $\sum_{m=0}^{5} mk\text{^}2$

- $\sum \mathbf{x_k}$ where k=1,2,3⋯5

Functions:

Syntax= function [Inputs]= my_function_name(Outputs)

Matlab Program for Summation and Multiplication of variables using function

Function [a,b]=myfunc(x,y,z)

a=x+y+z

```
b=x*y*z

end
```

Find a Maximum value of Column Vector using function

```
function [max_val max_loc]=my_max(v)

max_val= -inf

 for i=1:size(v)

        if v(i)>max_val

                max_val=v(i)

                max_loc=i

        end

 end
```

Task: Find the Average of x=1,2,3...100 with the help of function

Task: Find the Area of Sine wave from 0 to pi using function

Task: Find the Standard deviation

Plotting in MATLAB

The primary tool we will use for plotting in MATLAB is *plot()*.The MATLAB environment provides a wide variety of techniques to display data graphically.
The type of graph you choose to create depends on the nature of your data and what you want to reveal about the data. You can choose from many predefined graph types, such as line, stairs, stem, bar, histogram, and pie graphs as well as 3-D graphs, such as surfaces, slice planes, and streamlines. After you create a graph, you can extract specific information about the data. You can also edit graph components as well as annotate graphs. Annotations are the text, arrows, callouts, and other labels added to graphs to help viewers see what is important about the data. Lastly you can print your graph on any printer connected to your computer. Annotations can be added from the Insert menu like labels, titles, legend, line, arrows, text arrow, double arrow, text box, rectangle, ellipse, axes, light etc.

Using Plotting Tools and MATLAB Code

Plot a Sine Wave in Continuous Time:

```
>> t = 0:0.1:20;

>> y = sin(t);

>>plot(t,y)

>>xlabel('t')

>>ylabel('sin(t)')

>>grid on

>>title('plotting a sine wave')

>>axis([0 20 -2 2])
```

Plot a Sine Wave in Stairs Form:

```
>> t = 0:0.1:20;
>> y = sin(t);
>>stairs(t,y)
>>xlabel('t')
>>ylabel('sin(t)')
>>grid on
>>title('plotting a sine wave')
>>axis([0 20 -2 2])
```

Customization of plots

There are many commands used to customize plots by annotations, titles, axes labels, etc.

Explain the following commands in your own words.

xlabel

ylabel

title

grid

gtext

text

axis

figure

figure(n)

hold on

hold off

grid on

grid off

```
close(n)

subplot(a,b,c)

orient
```

Generation and Plotting of Signals

Single Plot

```
t = 0:0.1:20;
y1 = sin(t);
plot(t,y1,'-r*')
```

Single Plot with more details

```
t = 0:0.1:20;
y1 = sin(t);
plot(t,y1,'-r*','LineWidth',2,'MarkerEdgeColor','k','MarkerFaceColor',[.49 1 .63],'MarkerSize',10)
```

Multiple Plots in same Figure

```
t = 0:0.1:20;
y1 = sin(t);
y2=cos(t)
plot(t,y1,'-r*',t,y2,'b+')
```

Line Styles & Colours:-

Various line types, plot symbols, and colous may be obtained with plot(x,y,s) where s is a character string made from one element from any or all the following 3 columns:

Color		Marker Style		Line Style	
b	blue	.	point	-	solid
c	cyan	o	circle	:	dotted
g	green	x	x-mark	-.	dashdot
k	black	+	plus	--	dashed
m	magenta *	star			
r	red	s	square		

w	white	d	diamond
y	yellow	v	triangle (down)
		^	triangle (up)
		<	triangle (left)

- triangle (right)

p pentagram

h hexagram

For example, plot (X, Y, ‘g:’) plots a green dotted line and plot(X, Y,’rd’) plots red diamond at each data point. You may also edit colors and line styles directly from the menus which appear at the top of the figure windowThe default is to plot solid lines. A solid white line is produced by
>>plot (x, y, ’w-’)
The third argument is a string whose first character specifies the colour (optional) and the second the line style.

Multi–plots: -
Several graphs can be drawn on the same figure as:
>>plot (x, y, ’w-’, x, cos (2*pi*x), ’g--’)
A descriptive legend may be included with
>>legend (’Sin curve’, ’Cos curve’)
Which will give a list of line–styles, as they appeared in the plot command, followed by a brief description. MATLAB fits the legend in a suitable position, so as not to conceal the graphs whenever possible. For further information do help plot etc. The result of the commands
>>plot (x, y, ’w-’, x, cos (2*pi*x), ’g--’)
>>legend (’Sin curve’, ’Cos curve’)
>>title (’Multi-plot ’)
>>xlabel (’x axis’), ylabel (’y axis’)
>>grid

Hold: -
A call to plot clears the graphics window before plotting the current graph. This is not convenient if we wish to add further graphics to the figure at some later stage. To stop the window being cleared:
>>plot (x, y, ’w-’), hold

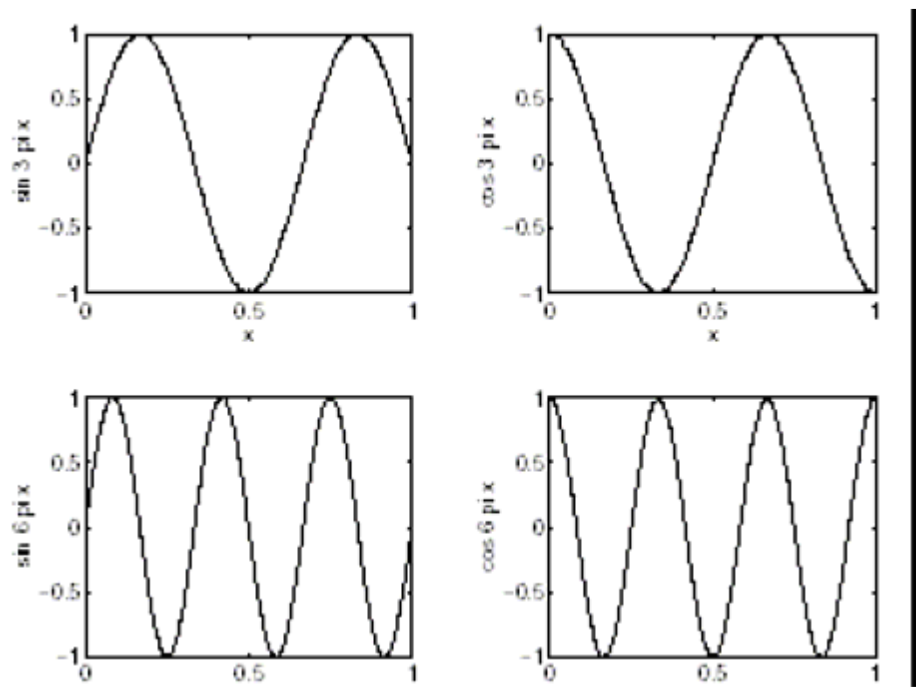
>>plot (x, y, ’gx’), hold off
“hold on” holds the current picture; “hold off” releases it (but does not clear the window, which can be done with clf). “hold” on its own toggles the hold state.

Subplot: -

The graphics window may be split into an m × n array of smaller windows into which we may plot one or more graphs. The windows are counted 1 to mn row–wise, starting from the top left. Both hold and grid work on the current subplot.

```
>>subplot (221), plot (x,y)
>>xlabel ('x'), ylabel ('sin 3 pi x')
>>subplot (222), plot (x, cos (3*pi*x))
>>xlabel ('x'), ylabel ('cos 3 pi x')
>>subplot (223), plot (x, sin (6*pi*x))
>>xlabel ('x'), ylabel ('sin 6 pi x')
>>subplot (224), plot (x, cos (6*pi*x))
>>xlabel ('x'), ylabel ('cos 6 pi x')
```

subplot(221) (or subplot(2,2,1)) specifies that the window should be split into a 2 × 2 array and we select the first subwindow.



Controlling Axes: -

If it is desired to plot a curve in a region specified by r = [x-min x-max y-min y-max]. Enter the command axis(r). This command sets the scaling to the prescribed limits. Find-out more about the command "axis" by typing:

```
help axis
```

If you wish to plot cos(t) of exercise 5 from time 0 to 10 seconds, and if you wish that the y-axis should be from –2 to 2, then enter the following command:

axis([0 10 -2 2])Once a plot has been created in the graphics window you may wish to change the range of x and y values shown on the picture

Property Editor

Figure properties can be changed interactively using thefollowing commands:

- **PlotEdit**
  Allows interactive changes to plots (add legend, lines, arrows, etc.)
- **PropEdit**
  Allows changes to all Handle Graphic properties in a MATLAB plot

Examples in Plotting

Example: Draw graphs of the functions

Task: y = sin(x)/x

Task: u= (1/(x-1)2)+x

Task: v= (x2+1)/(x2-4)

Task: ((10-x)1/3-1)/(4 - x2)1/2

PART C: Solving ordinary differential equations (ODE) symbolically

MATLAB has a command dsolve that solves ordinary differential equations (ODEs) symbolically.One form of the dsolve command is

dsolve(‘ODE’,’initial conditions’)

In the ODE string,D is used to represent the first derivative and Dn represents the nth derivative.

<u>Example</u>

To solve the differential equation y’(t) = ay(t) subject to the initial conditions y(0)=c and assign the solution to y,one give the commands

```
>>syms a c

>> y=dsolve('Dy=a*y','y(0)=c')

y =c*exp(a*t)
```

Solve the following ODE symbolically

Task: x”(t)+9x(t)=0 at x(0)=3 & x’(0) = 0

Task: y”(t) =-9y(t) at y(0) =1 & y’(0) = 0

Task: Y”(t) = -2y’(t) – y(t) at y(0) = 1 & y’(0) = 0

Task: Plot the sin, cosine and tan function in one window using different color and marker

Conclusion:

_____

_____

_____